Ejercicios APE – Ficheros en Java

Estudiante: Andres Vayas
Ingeniería en Software – 1ero B

Parte 1: APE 5 primera parte

Ejercicio 1. Crear un fichero en Java

Enunciado: Desarrollar un programa en Java que permita crear un archivo de texto llamado curso.txt.

Código

```
import java.io.File;
import java.io.IOException;

public class Ejercicio1 {
    public static void main(String[] args) {
        try {
            File archivo = new File("curso.txt");

            if (archivo.createNewFile()) {
                System.out.println("Archivo creado correctamente: "
                        + archivo.getName());
            } else {
                System.out.println("El archivo ya existe: "
                        + archivo.getName());
            }
        } catch (IOException e) {
            System.out.println("Ocurrió un error al crear el archivo: "
                    + e.getMessage());
        }
    }
}
```

Resultado esperado

- Primera ejecución: Archivo creado correctamente: curso.txt
- Siguientes ejecuciones: El archivo ya existe: curso.txt

Ejercicio 2. Escribir información en un fichero

Enunciado: Desarrollar un programa en Java que permita ingresar un mensaje por teclado y almacenarlo en un archivo de texto existente llamado estudiantes.txt.

Nota: el archivo estudiantes.txt debe existir previamente.

Código

```
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio2 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        System.out.print("Escribe un mensaje: ");
        String mensaje = teclado.nextLine();

        try {
            // El parámetro true conserva la información anterior
            FileWriter escritor = new FileWriter("estudiantes.txt", true);
            escritor.write(mensaje + System.lineSeparator());
            escritor.close();

            System.out.println("La información se guardó correctamente.");
        } catch (IOException e) {
            System.out.println("Ocurrió un error al escribir en el archivo: "
                    + e.getMessage());
        }

        teclado.close();
    }
}
```

Resultado esperado

- Se solicita el mensaje por teclado, se agrega al final de estudiantes.txt y se muestra: La información se guardó correctamente.

Ejercicio 3. Eliminar un fichero en Java

Enunciado: Desarrollar un programa en Java que permita eliminar el archivo de texto estudiantes.txt.

Código

```
import java.io.File;

public class Ejercicio3 {
    public static void main(String[] args) {
        File archivo = new File("estudiantes.txt");

        if (archivo.delete()) {
            System.out.println("El archivo " + archivo.getName()
                    + " se eliminó correctamente.");
        } else {
            System.out.println("No se pudo eliminar el archivo. "
                    + "Verifica que exista y que no esté en uso.");
        }
    }
}
```

Resultado esperado

- Si el archivo existe: El archivo estudiantes.txt se eliminó correctamente.
- Si no existe o no se puede borrar: se muestra el mensaje de error.

Parte 2: Ejercicios de APE clase 2 (05/11/2026)

Ejercicio 4. Registro de estudiantes y calificaciones

Enunciado: Desarrollar un programa en Java que permita registrar los datos de 3 estudiantes y guardar la información en un archivo llamado estudiantes.txt.

Requisitos: solicitar nombre y calificación, guardar ambos datos con cada estudiante en dos líneas, usar Scanner, BufferedWriter y FileWriter, y cerrar correctamente el archivo.

Código

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio4 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);

        // try-with-resources: cierra el archivo automáticamente al terminar
        try (BufferedWriter escritor =
                new BufferedWriter(new FileWriter("estudiantes.txt"))) {

            for (int i = 1; i <= 3; i++) {
                System.out.print("Nombre del estudiante " + i + ": ");
                String nombre = teclado.nextLine();

                System.out.print("Calificación de " + nombre + ": ");
                double calificacion = Double.parseDouble(teclado.nextLine());

                // Cada estudiante ocupa dos líneas
                escritor.write(nombre);
                escritor.newLine();
                escritor.write(String.valueOf(calificacion));
                escritor.newLine();
            }

            System.out.println("Datos guardados en estudiantes.txt");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("La calificación debe ser un número válido.");
        }

        teclado.close();
    }
}
```

Resultado esperado

- El archivo estudiantes.txt queda con 6 líneas: nombre y calificación de cada uno de los 3 estudiantes.

Ejercicio 5. Registro de productos con modo Append

Enunciado: Desarrollar un programa que permita registrar productos en un archivo llamado productos.txt.

Requisitos: solicitar nombre, precio y cantidad; calcular el valor total (precio × cantidad); guardar los resultados sin borrar los registros anteriores; usar BufferedWriter y modo append.

Código

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio5 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        String continuar;

        // El segundo parámetro true activa el modo append
        try (BufferedWriter escritor =
                new BufferedWriter(new FileWriter("productos.txt", true))) {

            do {
                System.out.print("Nombre del producto: ");
                String nombre = teclado.nextLine();

                System.out.print("Precio: ");
                double precio = Double.parseDouble(teclado.nextLine());

                System.out.print("Cantidad disponible: ");
                int cantidad = Integer.parseInt(teclado.nextLine());

                double total = precio * cantidad;

                escritor.write(String.format(Locale.US,
                    "Producto: %s | Precio: $%.2f | Cantidad: %d | Total: $%.2f",
                    nombre, precio, cantidad, total));
                escritor.newLine();

                System.out.print("¿Registrar otro producto? (si/no): ");
                continuar = teclado.nextLine();
            } while (continuar.equalsIgnoreCase("si"));

            System.out.println("Productos guardados en productos.txt");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Precio o cantidad no válidos.");
        }

        teclado.close();
    }
}
```

Resultado esperado

- Cada producto se agrega como una línea nueva en productos.txt, con su valor total calculado.
- Al ejecutar de nuevo el programa, los registros anteriores se conservan.

Ejercicio 6. Sistema de registro de ventas

Enunciado: Desarrollar un programa en Java que permita registrar las ventas de una tienda y guardar la información en un archivo llamado ventas.txt.

Requisitos: solicitar producto, precio y cantidad; calcular el subtotal; aplicar 10% de descuento si el subtotal es mayor o igual a $100; mostrar el total a pagar; repetir hasta que se escriba fin; guardar cada venta sin eliminar las anteriores y, al final, guardar el total acumulado.

Código

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;
import java.util.Locale;
import java.util.Scanner;

public class Ejercicio6 {
    public static void main(String[] args) {
        Scanner teclado = new Scanner(System.in);
        double totalAcumulado = 0;

        try (BufferedWriter escritor =
                new BufferedWriter(new FileWriter("ventas.txt", true))) {

            while (true) {
                System.out.print("Producto (escribe fin para terminar): ");
                String nombre = teclado.nextLine();
                if (nombre.equalsIgnoreCase("fin")) {
                    break;
                }

                System.out.print("Precio: ");
                double precio = Double.parseDouble(teclado.nextLine());

                System.out.print("Cantidad vendida: ");
                int cantidad = Integer.parseInt(teclado.nextLine());

                double subtotal = precio * cantidad;
                double descuento = 0;

                if (subtotal >= 100) {
                    descuento = subtotal * 0.10;
                }

                double totalPagar = subtotal - descuento;
                totalAcumulado += totalPagar;

                System.out.printf(Locale.US, "Total a pagar: $%.2f%n", totalPagar);

                escritor.write(String.format(Locale.US,
                    "Producto: %s | Subtotal: $%.2f | Descuento: $%.2f"
                    + " | Total: $%.2f",
                    nombre, subtotal, descuento, totalPagar));
                escritor.newLine();
            }

            escritor.write(String.format(Locale.US,
                "TOTAL ACUMULADO DE VENTAS: $%.2f", totalAcumulado));
            escritor.newLine();

            System.out.printf(Locale.US,
                "Total acumulado: $%.2f%n", totalAcumulado);
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } catch (NumberFormatException e) {
            System.out.println("Precio o cantidad no válidos.");
        }

        teclado.close();
```

```
    }
}
```

Resultado esperado

- Ejemplo: 3 unidades a $40 dan subtotal $120.00, descuento $12.00 y total a pagar $108.00.
- Cada venta se guarda en ventas.txt y al final se agrega la línea con el total acumulado.

Parte 3: Ejercicios APE 5 (segunda parte)

Ejercicio 7. Exploración y gestión de metadatos con File

Enunciado: Desarrollar un programa en Java que use la clase File para interactuar con la ruta "datos/prueba.txt".

Requisitos: comprobar con exists() si el recurso existe; si no existe, crearlo con createNewFile() manejando IOException con try-catch; y mostrar por consola getName() y getPath(), tanto si ya existía como si se acaba de crear.

Código

```
import java.io.File;
import java.io.IOException;

public class Ejercicio7 {
    public static void main(String[] args) {
        File archivo = new File("datos/prueba.txt");

        // 1. Comprobación previa
        if (archivo.exists()) {
            System.out.println("El archivo ya existe.");
        } else {
            try {
                // La carpeta "datos" debe existir; si no, se crea
                archivo.getParentFile().mkdirs();

                // 2. Creación controlada
                if (archivo.createNewFile()) {
                    System.out.println("Archivo creado correctamente.");
                }
            } catch (IOException e) {
                System.out.println("Error de E/S al crear el archivo: "
                        + e.getMessage());
            }
        }

        // 3. Extracción de información
        System.out.println("Nombre: " + archivo.getName());
        System.out.println("Ruta: " + archivo.getPath());
    }
}
```

Resultado esperado

- Primera ejecución: Archivo creado correctamente. / Nombre: prueba.txt / Ruta: datos/prueba.txt (en Windows: datos\prueba.txt).
- Siguientes ejecuciones: El archivo ya existe. y se muestran el mismo nombre y ruta.

Ejercicio 8. Escritura estructurada por líneas con BufferedWriter

Enunciado: Desarrollar un programa en Java que cree un fichero de texto llamado materias.txt y escriba en él cinco asignaturas.

Requisitos: usar BufferedWriter enlazado con FileWriter, newLine() para una asignatura por línea, cerrar el flujo con close() y controlar IOException.

Código

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio8 {
    public static void main(String[] args) {
        String[] materias = {
            "Programación Orientada a Objetos",
            "Cálculo Diferencial",
            "Estructuras de Datos",
            "Bases de Datos",
            "Fundamentos de Redes"
        };

        BufferedWriter escritor = null;
        try {
            escritor = new BufferedWriter(new FileWriter("materias.txt"));

            for (String materia : materias) {
                escritor.write(materia);
                escritor.newLine(); // una asignatura por línea
            }
            System.out.println("materias.txt creado correctamente.");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } finally {
            // Liberar recursos con close()
            try {
                if (escritor != null) {
                    escritor.close();
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el archivo: " + e.getMessage());
            }
        }
    }
}
```

Resultado esperado

- Se genera materias.txt en la carpeta del proyecto con 5 líneas, una asignatura por línea.
- Verificación: abrir el archivo y comprobar que cada asignatura está en una línea independiente.

Ejercicio 9. Control del modo de apertura (append vs sobrescribir)

Enunciado: A partir del archivo materias.txt del ejercicio anterior, desarrollar un programa que añada dos asignaturas nuevas al final sin borrar el contenido previo.

Requisitos: abrir FileWriter con append = true, usar newLine() para mantener el formato de líneas y cerrar el flujo con close().

Código

```
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.io.IOException;

public class Ejercicio9 {
    public static void main(String[] args) {
        BufferedWriter escritor = null;
        try {
            // El segundo parámetro true activa el modo append
            escritor = new BufferedWriter(new FileWriter("materias.txt", true));

            escritor.write("Ingeniería de Software");
            escritor.newLine();
            escritor.write("Sistemas Operativos");
            escritor.newLine();

            System.out.println("Asignaturas añadidas al final de materias.txt");
        } catch (IOException e) {
            System.out.println("Error al escribir el archivo: " + e.getMessage());
        } finally {
            try {
                if (escritor != null) {
                    escritor.close();
                }
            } catch (IOException e) {
                System.out.println("Error al cerrar el archivo: " + e.getMessage());
            }
        }
    }
}
```

Resultado esperado

- materias.txt conserva las 5 asignaturas originales y ahora tiene 2 nuevas al final (7 líneas en total).
- Si se usara FileWriter sin el parámetro true, el contenido anterior se sobrescribiría.

Ejercicio 10. Ciclo de vida completo y borrado seguro

Enunciado: Desarrollar un programa en Java que gestione el ciclo de vida del archivo "datos/temporal.txt": comprobar si existe, crearlo si hace falta y eliminarlo solo si el usuario confirma.

Requisitos: crear un objeto File con la ruta, usar exists() y createNewFile() controlando IOException, declarar boolean confirmarBorrado = true para simular la confirmación, y usar delete() solo si es verdadera mostrando su resultado.

Código

```
import java.io.File;
import java.io.IOException;

public class Ejercicio10 {
    public static void main(String[] args) {
        File archivo = new File("datos/temporal.txt");

        // Consulta del estado y creación segura
        if (!archivo.exists()) {
            try {
                archivo.getParentFile().mkdirs(); // asegura la carpeta "datos"
                archivo.createNewFile();
                System.out.println("Archivo creado: " + archivo.getPath());
            } catch (IOException e) {
                System.out.println("Error de E/S al crear el archivo: "
                        + e.getMessage());
            }
        } else {
            System.out.println("El archivo ya existe: " + archivo.getPath());
        }

        // Validación: simula la confirmación del usuario
        boolean confirmarBorrado = true;

        if (confirmarBorrado) {
            boolean eliminado = archivo.delete();
            System.out.println("Resultado de la eliminación: " + eliminado);
        } else {
            System.out.println("Borrado cancelado por el usuario.");
        }
    }
}
```

Resultado esperado

- Si el archivo no existe, se crea y luego se elimina: Resultado de la eliminación: true.
- Con confirmarBorrado = false, el archivo se conserva y se muestra: Borrado cancelado por el usuario.

Repositorio de GitHub

Enlace: https://github.com/Andyyyxx27/ficherosss